ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ассистент		В. В. Жукалин
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №7

СОЕДИНЕНИЕ И ТЕСТИРОВАНИЕ КЛИЕНТСКОЙ И СЕРВЕРНОЙ ЧАСТЕЙ ВЕБ-ПРИЛОЖЕНИЯ

по курсу: Web-программирование

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4321		И.А. Лаврешин
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Соединить клиентские компоненты SignalBoard с серверными Route Handlers и PostgreSQL, заменить демонстрационное состояние пользователя и отслеживания реальными запросами, реализовать состояния загрузки и ошибок, проверить HTTP-контракты и авторизацию, добавить модульные и end-to-end тесты и выполнить итоговую проверку всего приложения.

2 Задание

Определить HTTP-контракт между клиентом и сервером, реализовать GET /api/signals с преобразованием Decimal-координат и вычислением isTracked для текущего пользователя. Заменить DemoAuthProvider на AuthProvider, восстанавливать сессию через GET /api/auth/me, подключить формы регистрации, входа и выхода к API, получать галерею из PostgreSQL, реализовать loading, error и empty состояния, сохранять отслеживание через POST и DELETE, обрабатывать 400, 401, 404, 409 и 500, не доверять userId и isTracked от клиента. Сохранить модальное окно, карту, меню и адаптивность. Установить Vitest и написать модульные тесты нормализации и валидации, установить Playwright и написать сквозной сценарий регистрации и отслеживания, выполнить итоговую проверку validate, migrate status, lint, unit, build и e2e.

3 Ход выполнения задания

Схема прохождения запроса: Client Component (SignalsGallery, AuthProvider) → fetch с cookie signal_session → Route Handler app/api/.../route.ts → проверка сессии (readSession, getCurrentUser) → Prisma Client → PostgreSQL → DTO в JSON → обновление состояния React.

```
Браузер (Client Components)            Next.js server
PostgreSQL
──────────────────────────             ──────────────
──────────

AuthProvider    ── GET /api/auth/me ──▶ route.ts – getCurrentUser ─▶ User
SignalsGallery  ── GET /api/signals ──▶ route.ts – prisma.findMany ─▶
NeighborSignal
                ── POST/DELETE ──────▶ /api/favorites/[id] ──────▶
TrackedSignal
                ◀── JSON DTO ─────────
```

Таблица 1 – HTTP-контракты

Метод и путь	Тело запроса	Успех	Ошибки
GET /api/signals	—	200 {signals: NeighborSignalDTO[]}	500
GET /api/auth/me	—	200 {user: UserDTO | null}	—
POST /api/auth/register	name, email, password, confirmPassword	201 {user}	400, 409, 500
POST /api/auth/login	email, password	200 {user}	400, 401, 500
POST /api/auth/logout	—	200 {ok: true}	—
POST /api/favorites/{signalId}	—	201 {favorite}	401, 404
DELETE /api/favorites/{signalId}	—	200 {deleted}	401

UserDTO содержит только id, name и email: passwordHash, createdAt и updatedAt клиенту не нужны, а хеш пароля нельзя раскрывать. NeighborSignalDTO содержит id, slug, title, summary, category, location, image, imageAlt, latitude, longitude и isTracked. Координаты в базе имеют тип Decimal, который Prisma возвращает объектом Decimal.js; JSON.stringify превратил бы его в строку, поэтому сервер явно преобразует значения через Number(). Поле isTracked вычисляется на сервере по строкам TrackedSignal пользователя из проверенной сессии и не принимается от клиента.

Маршрут app/api/signals/route.ts:

```
// Возвращает локальные сигналы и персональное состояние отслеживания.
```

```
import { NextResponse } from "next/server";
import { getCurrentUser } from "@/app/lib/current-user";
import { prisma } from "@/app/lib/prisma";

// Ответ зависит от cookie текущего запроса и не является общим кэшем.
export const dynamic = "force-dynamic";

export async function GET() {
  try {
    // Гость получает сигналы без персональных связей.
    const user = await getCurrentUser();

    // Читаем предметные данные и только нужные tracked signalId.
    const [signals, tracked] = await Promise.all([
      prisma.neighborSignal.findMany({ orderBy: { title: "asc" } }),
      user
        ? prisma.trackedSignal.findMany({
            where: { userId: user.id },
            select: { signalId: true },
          })
        : Promise.resolve([]),
    ]);

    // Set обеспечивает быструю проверку каждого сигнала.
    const trackedIds = new Set(tracked.map((item) => item.signalId));

    // Формируем стабильный DTO без внутренних временных полей.
    const result = signals.map((signal) => ({
      id: signal.id,
      slug: signal.slug,
      title: signal.title,
      summary: signal.summary,
      category: signal.category,
      location: signal.location,
      image: signal.image,
      imageAlt: signal.imageAlt,
      latitude: Number(signal.latitude),
      longitude: Number(signal.longitude),
      isTracked: trackedIds.has(signal.id),
    }));

    return NextResponse.json({ signals: result });
  } catch {
    // Не раскрываем клиенту объект ошибки подключения или DATABASE_URL.
    return NextResponse.json(
      { error: "Не удалось загрузить сигналы" },
      { status: 500 },
    );
  }
}
```

AuthProvider заменил DemoAuthProvider в корневом layout. При монтировании он запрашивает GET /api/auth/me и находится в состоянии loading, пока ответ не получен: без этого интерфейс на мгновение показал бы гостя вошедшему пользователю. Cookie httpOnly недоступна JavaScript, поэтому состояние восстанавливается только запросом к серверу; браузер сам

прикладывает cookie к fetch с credentials: "same-origin". Функции login, register и logout вызывают соответствующие маршруты и обновляют user только после успешного ответа:

```
// Восстанавливаем состояние из серверной cookie, не читая её на клиенте.
const fetchCurrentUser = useCallback(async (): Promise<UserDTO | null> => {
  const response = await fetch("/api/auth/me", {
    cache: "no-store",
    credentials: "same-origin",
  });

  if (!response.ok) {
    return null;
  }

  const payload = (await response.json()) as { user: UserDTO | null };
  return payload.user;
}, []);

const refresh = useCallback(async () => {
  const currentUser = await fetchCurrentUser();

  setUser(currentUser);
  setStatus(currentUser ? "authenticated" : "guest");
}, [fetchCurrentUser]);

useEffect(() => {
  void fetchCurrentUser().then((currentUser) => {
    setUser(currentUser);
    setStatus(currentUser ? "authenticated" : "guest");
  });
}, [fetchCurrentUser]);

const login = useCallback(async (email: string, password: string) => {
  const response = await fetch("/api/auth/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    credentials: "same-origin",
    body: JSON.stringify({ email, password }),
  });

  if (!response.ok) throw new Error(await readError(response));

  const payload = (await response.json()) as { user: UserDTO };
  setUser(payload.user);
  setStatus("authenticated");
}, []);

const register
```

Формы входа и регистрации передают данные в AuthProvider, блокируют кнопку на время запроса и показывают сообщение сервера при ошибке:

```
async function handleSubmit(event: FormEvent<HTMLFormElement>) {
  event.preventDefault();
  setError("");
```

```
    setPending(true);

    const form = new FormData(event.currentTarget);

    try {
      // Передаём пароль серверу только в теле HTTPS-запроса production-
среды.
      await login(String(form.get("email") ?? ""),
String(form.get("password") ?? ""));
      router.push("/gallery");
    } catch (value) {
      setError(value instanceof Error ? value.message : "Не удалось войти");
    } finally {
      setPending(false);
    }
  }
```

Загрузка галереи. SignalsGallery запрашивает /api/signals в useEffect. AbortController отменяет запрос при размонтировании или повторной загрузке, чтобы устаревший ответ не перезаписал состояние. Реализованы три состояния: loading (role="status" «Загрузка сигналов…»), error (role="alert" с кнопкой «Повторить») и empty («Сигналы пока не добавлены.»). Loading означает, что ответа ещё нет, empty – что ответ получен, но массив пуст.

```
  useEffect(() => {
    const controller = new AbortController();

    async function loadSignals() {
      setLoading(true);
      setError("");

      try {
        const response = await fetch("/api/signals", {
          cache: "no-store",
          credentials: "same-origin",
          signal: controller.signal,
        });

        if (!response.ok) throw new Error("Не удалось загрузить сигналы");

        const payload = (await response.json()) as { signals:
NeighborSignalDTO[] };
        setSignals(payload.signals);
      } catch (value) {
        // Игнорируем ожидаемую отмену при размонтировании компонента.
        if (value instanceof DOMException && value.name === "AbortError")
return;
        setError(value instanceof Error ? value.message : "Ошибка загрузки");
      } finally {
        if (!controller.signal.aborted) setLoading(false);
      }
    }

    void loadSignals();

    // Отменяем запрос, если компонент больше не используется.
```

```
    return () => controller.abort();
  }, [reloadKey, status]);
```

Переключение отслеживания выполняется пессимистично: интерфейс меняет карточку только после того, как сервер подтвердил POST или DELETE. При ответе 401 AuthProvider обновляется и пользователь видит сообщение о завершении сеанса. userId в запрос не передаётся – сервер берёт его из подписанной сессии:

```
  async function toggleTracked(signal: NeighborSignalDTO) {
    if (status !== "authenticated") return;

    setPendingId(signal.id);
    setError("");

    try {
      const response = await fetch(`/api/favorites/${signal.id}`, {
        method: signal.isTracked ? "DELETE" : "POST",
        credentials: "same-origin",
      });

      if (response.status === 401) {
        // Синхронизируем UI, если сервер больше не принимает сессию.
        await refresh();
        throw new Error("Сеанс завершён. Выполните вход снова.");
      }

      if (!response.ok) throw new Error("Не удалось изменить отслеживание");

      const nextTracked = !signal.isTracked;

      // Меняем только карточку, изменение которой подтвердил сервер.
      setSignals((current) =>
        current.map((item) =>
          item.id === signal.id
            ? { ...item, isTracked: nextTracked }
            : item,
        ),
      );
      setSelected((current) =>
        current && current.id === signal.id
          ? { ...current, isTracked: nextTracked }
          : current,
      );
    } catch (value) {
      setError(value instanceof Error ? value.message : "Ошибка
отслеживания");
    } finally {
      setPendingId(null);
    }
  }
```

Обработка ошибки сервера на клиенте проверена при недоступной базе данных: GET /api/signals вернул 500 без раскрытия деталей подключения

(рисунок 1), а галерея показала состояние ошибки с кнопкой «Повторить» (рисунок 2).

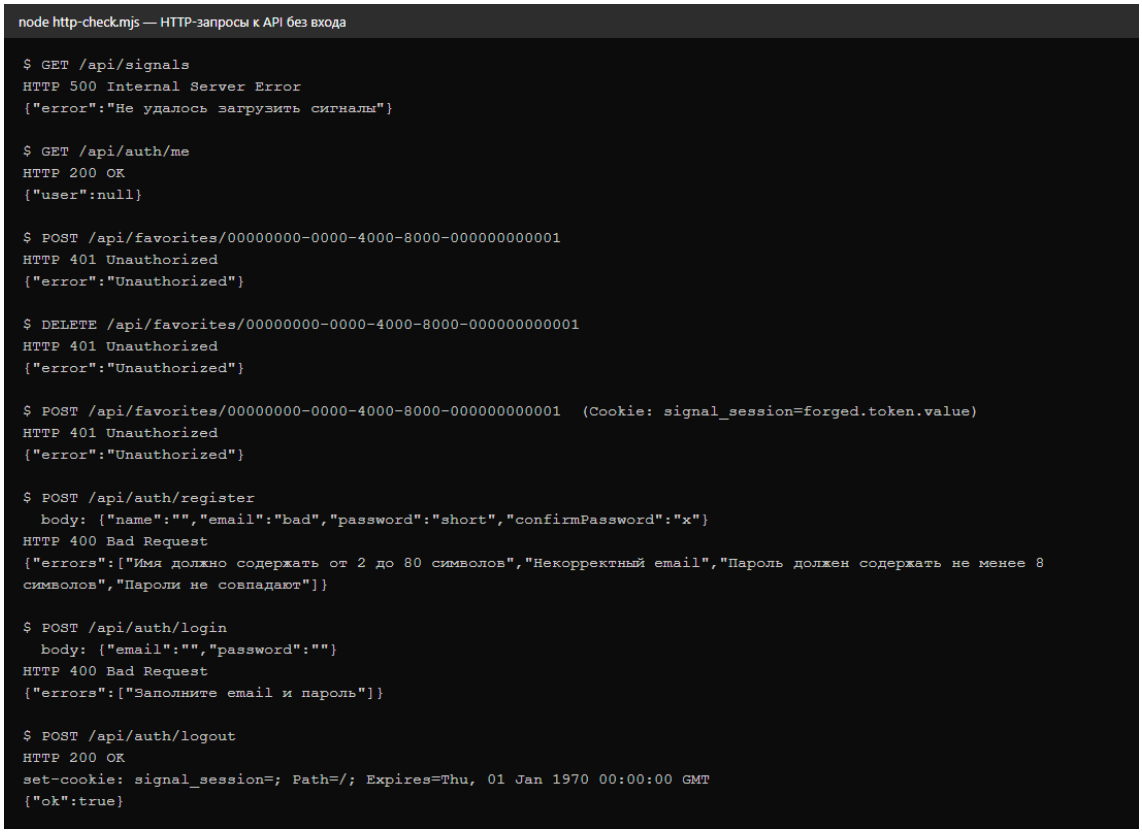


Рисунок 1 – HTTP-ответы API без входа и без базы данных

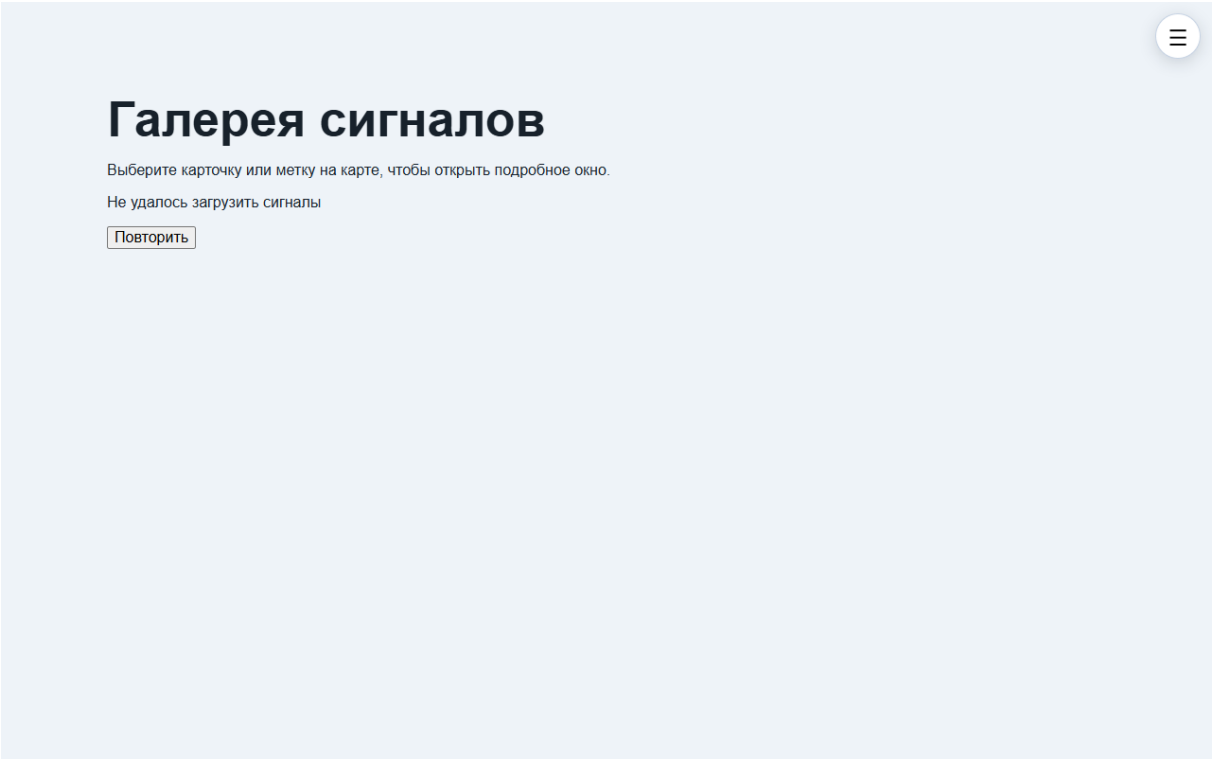


Рисунок 2 – Состояние error галереи при ответе 500

[ВСТАВИТЬ СКРИНШОТ: /gallery после входа neighbor@example.test – карточки с кнопками «Добавить в отслеживание»]

Рисунок 3 – Галерея с данными из PostgreSQL после входа

[ВСТАВИТЬ СКРИНШОТ: отметить сигнал, нажать F5 – кнопка «Удалить из отслеживания» осталась]

Рисунок 4 – Сохранение отслеживания после перезагрузки

[ВСТАВИТЬ СКРИНШОТ: DevTools → Network: GET /api/signals 200 и POST /api/favorites/{id} 201; Response без passwordHash]

Рисунок 5 – Вкладка Network без секретов

[ВСТАВИТЬ СКРИНШОТ: psql: SELECT u.email, s.slug FROM "TrackedSignal" t JOIN "User" u ON u.id=t."userId" JOIN "NeighborSignal" s ON s.id=t."signalId";]

Рисунок 6 – SQL-проверка TrackedSignal

Таблица 2 – Матрица ручных сценариев

Сценарий	**Ожидаемый результат**	**Статус**
Гость открывает /gallery	Карточки и ссылка «Войдите, чтобы отслеживать сигнал»	*[проверить с БД]*
База недоступна	500 и состояние error с кнопкой «Повторить»	Проверено
Гость: POST /api/favorites/{id}	401	Проверено
Поддельная cookie	401	Проверено
Регистрация с ошибками полей	400 и сообщение в форме	Проверено (API)
Регистрация занятого email	409 «Этот email уже зарегистрирован»	*[проверить с БД]*

Вход с неверным паролем	401 «Неверный email или пароль»	*[проверить с БД]*
Вход seed-пользователя	Переход в /gallery, имя в меню	*[проверить с БД]*
Отслеживание и reload	Состояние сохраняется после F5	*[проверить с БД]*
Выход	Cookie удалена, интерфейс гостя	Проверено (API)

Модульные тесты. Vitest настроен на окружение node и каталог tests/unit; плагин vite-tsconfig-paths подключает алиас @/. Тесты проверяют чистые функции normalizeEmail и validateRegistration и не требуют базы данных:

```
// Настраивает быстрые unit tests чистой серверной логики.

import { defineConfig } from "vitest/config";
import tsconfigPaths from "vite-tsconfig-paths";

export default defineConfig({
  // Поддерживаем алиасы путей из tsconfig.json.
  plugins: [tsconfigPaths()],
  test: {
    // Чистая валидация не требует DOM или браузера.
    environment: "node",
    include: ["tests/unit/**/*.test.ts"],
  },
});
// Проверяет чистую нормализацию и server-side validation без базы.

import { describe, expect, test } from "vitest";
import {
  normalizeEmail,
  validateRegistration,
} from "@/app/lib/validation";

describe("normalizeEmail", () => {
  test("удаляет пробелы и приводит email к нижнему регистру", () => {
    expect(normalizeEmail("  Neighbor@Example.TEST ")).toBe(
      "neighbor@example.test",
    );
  });
});

describe("validateRegistration", () => {
  test("принимает корректный набор полей", () => {
    const result = validateRegistration({
      name: "Сосед",
      email: "neighbor@example.test",
      password: "SignalPass_2026!",
      confirmPassword: "SignalPass_2026!",
    });

    expect(result.errors).toEqual([]);
    expect(result.data?.email).toBe("neighbor@example.test");
  });

  test("отклоняет короткий пароль", () => {
```

```
    const result = validateRegistration({
      name: "Сосед",
      email: "neighbor@example.test",
      password: "short",
      confirmPassword: "short",
    });

    expect(result.data).toBeNull();
    expect(result.errors).toContain(
      "Пароль должен содержать не менее 8 символов",
    );
  });

  test("отклоняет несовпадающее подтверждение", () => {
    const result = validateRegistration({
      name: "Сосед",
      email: "neighbor@example.test",
      password: "SignalPass_2026!",
      confirmPassword: "Different_2026!",
    });

    expect(result.errors).toContain("Пароли не совпадают");
  });
});
```

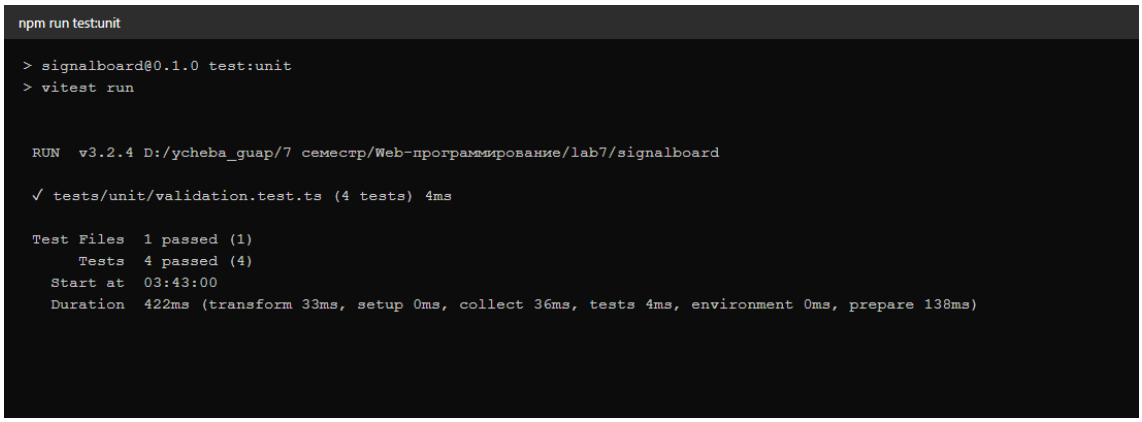


Рисунок 7 – Результат npm run test:unit: 4 теста пройдены

End-to-end тест. Playwright запускает production-сервер (npm run start) через webServer и открывает страницы по baseURL. Используется один worker, потому что все тесты работают с одной локальной базой. Сценарий регистрирует пользователя с уникальным email вида e2e-<время>@example.test, добавляет сигнал в отслеживание, перезагружает страницу и проверяет, что состояние прочитано из PostgreSQL, затем выходит. В afterAll созданный пользователь удаляется, а его строки TrackedSignal удаляются каскадно, поэтому повторный запуск не накапливает тестовые данные.

```
// Запускает E2E против production-сборки SignalBoard.
```

```
import "dotenv/config";
import { defineConfig, devices } from "@playwright/test";

export default defineConfig({
  testDir: "./tests/e2e",
  // Общая локальная база проверяется последовательно.
  workers: 1,
  use: {
    baseURL: "http://127.0.0.1:3000",
    trace: "on-first-retry",
    ...devices["Desktop Chrome"],
  },
  webServer: {
    // Перед test:e2e уже должна существовать npm run build.
    command: "npm run start",
    url: "http://127.0.0.1:3000",
    reuseExistingServer: !process.env.CI,
    timeout: 120_000,
  },
});
// Проверяет регистрацию, cookie-сессию и сохранение отслеживания после
reload.

import "dotenv/config";
import { PrismaClient } from "@prisma/client";
import { expect, test } from "@playwright/test";

const prisma = new PrismaClient();
let testEmail = "";

test.afterAll(async () => {
  // Удаляем только пользователя, созданного этим тестом.
  if (testEmail) {
    await prisma.user.deleteMany({ where: { email: testEmail } });
  }
  await prisma.$disconnect();
});

test("пользователь регистрируется и сохраняет сигнал в отслеживание", async
({
  page,
}) => {
  // Уникальный email исключает конфликт с предыдущим локальным запуском.
  testEmail = `e2e-${Date.now()}@example.test`;

  await page.goto("/auth/register");
  await page.getByLabel("Имя пользователя").fill("E2E сосед");
  await page.getByLabel("Email").fill(testEmail);
  await page.getByLabel("Пароль", { exact: true }).fill("SignalPass_2026!");
  await page.getByLabel("Повторите пароль").fill("SignalPass_2026!");
  await page.getByRole("button", { name: "Зарегистрироваться" }).click();

  // После успешного API-ответа форма переводит пользователя в галерею.
  await expect(page).toHaveURL(/\/gallery$/);

  const addTracked = page.getByRole("button", {
    name: "Добавить в отслеживание: Ищу ключи от подъезда",
  });
  await addTracked.click();

  // Сервер подтвердил изменение, поэтому доступно действие удаления.
```

```
  await expect(
    page.getByRole("button", {
      name: "Удалить из отслеживания: Ищу ключи от подъезда",
    }),
  ).toBeVisible();

  // Перезагрузка повторно читает cookie и TrackedSignal из PostgreSQL.
  await page.reload();
  await expect(
    page.getByRole("button", {
      name: "Удалить из отслеживания: Ищу ключи от подъезда",
    }),
  ).toBeVisible();

  // Завершаем сессию через тот же browser context.
  await page.request.post("/api/auth/logout");
  await page.reload();
  await expect(
    page.getByRole("link", { name: "Войдите, чтобы отслеживать сигнал"
}).first(),
  ).toBeVisible();
});
```

Итоговая проверка. Команды, не требующие базы данных, выполнены: prisma validate – схема валидна, npm run lint – без ошибок, npm run test:unit – 4 из 4, npm run build – успешно (рисунки 8–9). Команды prisma migrate status и npm run test:e2e выполняются при запущенном PostgreSQL.

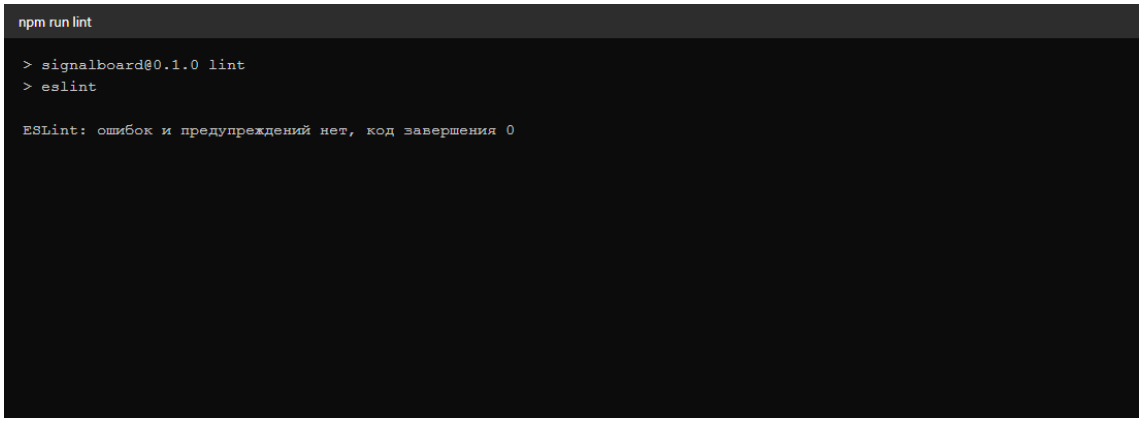


Рисунок 8 – Результат npm run lint

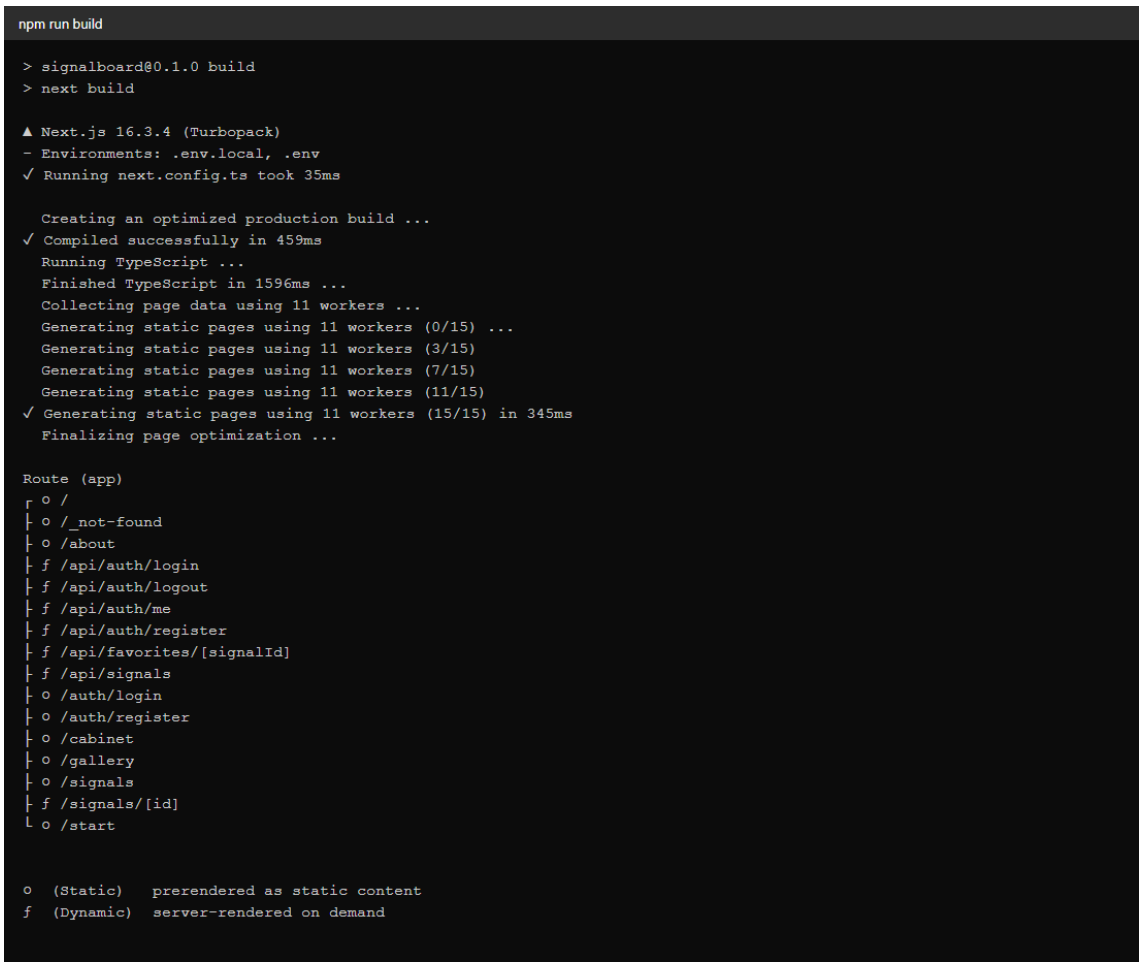


Рисунок 9 – Результат npm run build

[ВСТАВИТЬ СКРИНШОТ: npm run db:status – «Database schema is up to date!»]

Рисунок 10 – Результат prisma migrate status

[ВСТАВИТЬ СКРИНШОТ: после npm run build: «1 passed»]

Рисунок 11 – Результат npm run test:e2e

Повторная ручная проверка ЛР3–ЛР4 на production-сборке ЛР7: на ширинах 320, 375, 768, 1024 и 1440 CSS px маршруты /start, /gallery, /auth/login и /auth/register не имеют горизонтальной прокрутки; бургер-меню открывается, закрывается клавишей Escape и автоматически закрывается

после перехода по ссылке (рисунок 12). Модальное окно и карта используют данные из API и проверяются после запуска базы.

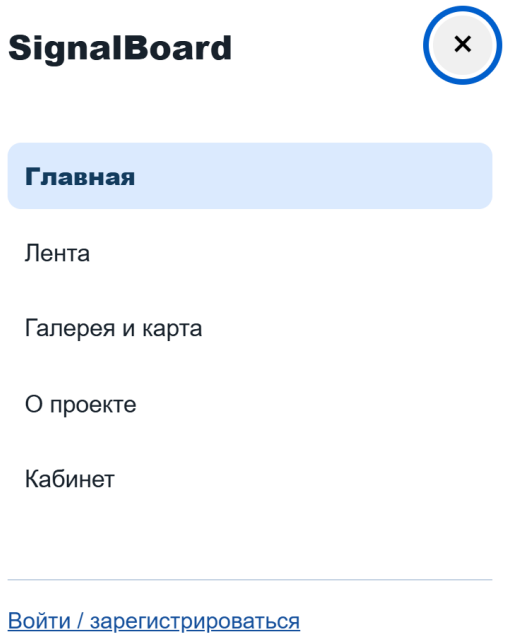


Рисунок 12 – Бургер-меню ЛР7 на ширине 375 CSS px

4 Вывод

В ходе работы клиентская часть SignalBoard соединена с серверной: галерея получает сигналы из PostgreSQL через GET /api/signals, AuthProvider восстанавливает сессию из httpOnly-cookie через /api/auth/me, формы входа и регистрации работают через API, отслеживание сохраняется POST и DELETE с серверной проверкой пользователя. Реализованы состояния loading, error и empty, отмена запросов AbortController и обработка 400, 401, 404, 409 и 500. Модульные тесты Vitest проверяют валидацию независимо от базы, сценарий Playwright проверяет регистрацию и сохранение отслеживания на production-сборке и удаляет тестовые данные.

Итог по курсу. За семь лабораторных работ SignalBoard прошёл путь от каркаса Next.js до полноценного приложения: маршрутизация App Router, интерактивная клиентская часть с доступными диалогами и картой, адаптивная вёрстка с проверкой Lighthouse, реляционная база PostgreSQL с Prisma, серверная авторизация с хешированием паролей и подписанной сессией, соединение клиента с API и автоматическое тестирование.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

Файл app/api/signals/route.ts

```
// Возвращает локальные сигналы и персональное состояние отслеживания.

import { NextResponse } from "next/server";
import { getCurrentUser } from "@/app/lib/current-user";
import { prisma } from "@/app/lib/prisma";

// Ответ зависит от cookie текущего запроса и не является общим кэшем.
export const dynamic = "force-dynamic";

export async function GET() {
  try {
    // Гость получает сигналы без персональных связей.
    const user = await getCurrentUser();

    // Читаем предметные данные и только нужные tracked signalId.
    const [signals, tracked] = await Promise.all([
      prisma.neighborSignal.findMany({ orderBy: { title: "asc" } }),
      user
        ? prisma.trackedSignal.findMany({
            where: { userId: user.id },
            select: { signalId: true },
          })
        : Promise.resolve([]),
    ]);

    // Set обеспечивает быструю проверку каждого сигнала.
    const trackedIds = new Set(tracked.map((item) => item.signalId));

    // Формируем стабильный DTO без внутренних временных полей.
    const result = signals.map((signal) => ({
      id: signal.id,
      slug: signal.slug,
      title: signal.title,
      summary: signal.summary,
      category: signal.category,
      location: signal.location,
      image: signal.image,
      imageAlt: signal.imageAlt,
      latitude: Number(signal.latitude),
      longitude: Number(signal.longitude),
      isTracked: trackedIds.has(signal.id),
    }));

    return NextResponse.json({ signals: result });
  } catch {
    // Не раскрываем клиенту объект ошибки подключения или DATABASE_URL.
    return NextResponse.json(
      { error: "Не удалось загрузить сигналы" },
      { status: 500 },
    );
  }
}
```

Файл app/providers/AuthProvider.tsx

```
// Синхронизирует клиентский интерфейс с серверной cookie-сессией.
```

```
"use client";

import {
  createContext,
  useCallback,
  useContext,
  useEffect,
  useMemo,
  useState,
  type ReactNode,
} from "react";

export type UserDTO = {
  id: string;
  name: string;
  email: string;
};

type AuthStatus = "loading" | "authenticated" | "guest";

type AuthContextValue = {
  user: UserDTO | null;
  status: AuthStatus;
  refresh: () => Promise<void>;
  login: (email: string, password: string) => Promise<void>;
  register: (input: {
    name: string;
    email: string;
    password: string;
    confirmPassword: string;
  }) => Promise<void>;
  logout: () => Promise<void>;
};

const AuthContext = createContext<AuthContextValue | null>(null);

// Читаем безопасное сообщение из согласованного JSON-ответа.
async function readError(response: Response): Promise<string> {
  const payload = (await response.json().catch(() => null)) as
    | { error?: string; errors?: string[] }
    | null;

  return payload?.errors?.[0] ?? payload?.error ?? "Ошибка запроса";
}

export default function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<UserDTO | null>(null);
  const [status, setStatus] = useState<AuthStatus>("loading");

  // Восстанавливаем состояние из серверной cookie, не читая её на клиенте.
  const fetchCurrentUser = useCallback(async (): Promise<UserDTO | null> => {
    const response = await fetch("/api/auth/me", {
      cache: "no-store",
      credentials: "same-origin",
    });

    if (!response.ok) {
      return null;
    }
```

```
  const payload = (await response.json()) as { user: UserDTO | null };
  return payload.user;
}, []);

const refresh = useCallback(async () => {
  const currentUser = await fetchCurrentUser();

  setUser(currentUser);
  setStatus(currentUser ? "authenticated" : "guest");
}, [fetchCurrentUser]);

useEffect(() => {
  void fetchCurrentUser().then((currentUser) => {
    setUser(currentUser);
    setStatus(currentUser ? "authenticated" : "guest");
  });
}, [fetchCurrentUser]);

const login = useCallback(async (email: string, password: string) => {
  const response = await fetch("/api/auth/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    credentials: "same-origin",
    body: JSON.stringify({ email, password }),
  });

  if (!response.ok) throw new Error(await readError(response));

  const payload = (await response.json()) as { user: UserDTO };
  setUser(payload.user);
  setStatus("authenticated");
}, []);

const register = useCallback(async (input: {
  name: string;
  email: string;
  password: string;
  confirmPassword: string;
}) => {
  const response = await fetch("/api/auth/register", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    credentials: "same-origin",
    body: JSON.stringify(input),
  });

  if (!response.ok) throw new Error(await readError(response));

  const payload = (await response.json()) as { user: UserDTO };
  setUser(payload.user);
  setStatus("authenticated");
}, []);

const logout = useCallback(async () => {
  const response = await fetch("/api/auth/logout", {
    method: "POST",
    credentials: "same-origin",
  });
```

```
    if (!response.ok) throw new Error(await readError(response));
    setUser(null);
    setStatus("guest");
  }, []);

  // Стабилизируем объект контекста для дочерних Client Components.
  const value = useMemo(
    () => ({ user, status, refresh, login, register, logout }),
    [user, status, refresh, login, register, logout],
  );

  return <AuthContext.Provider
value={value}>{children}</AuthContext.Provider>;
}

export function useAuth(): AuthContextValue {
  const value = useContext(AuthContext);
  if (!value) throw new Error("useAuth must be used inside AuthProvider");
  return value;
}
```

Файл app/auth/login/page.tsx

```
// Подключает форму входа к POST /api/auth/login через AuthProvider.

"use client";

import Link from "next/link";
import { useRouter } from "next/navigation";
import { useState, type FormEvent } from "react";
import { useAuth } from "@/app/providers/AuthProvider";

export default function LoginPage() {
  const router = useRouter();
  const { login } = useAuth();
  const [error, setError] = useState("");
  const [pending, setPending] = useState(false);

  async function handleSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault();
    setError("");
    setPending(true);

    const form = new FormData(event.currentTarget);

    try {
      // Передаём пароль серверу только в теле HTTPS-запроса production-
среды.
      await login(String(form.get("email") ?? ""),
String(form.get("password") ?? ""));
      router.push("/gallery");
    } catch (value) {
      setError(value instanceof Error ? value.message : "Не удалось войти");
    } finally {
      setPending(false);
    }
  }

  return (
    <main className="form-page">
      <h1>Вход</h1>
      <form className="auth-form" onSubmit={handleSubmit}>
```

```
        <label htmlFor="email">Email</label>
        <input
          id="email"
          name="email"
          type="email"
          autoComplete="email"
          required
        />

        <label htmlFor="password">Пароль</label>
        <input
          id="password"
          name="password"
          type="password"
          autoComplete="current-password"
          minLength={8}
          required
        />

        {error ? <p role="alert" className="form-error">{error}</p> : null}
        <button type="submit" disabled={pending}>
          {pending ? "Вход…" : "Войти"}
        </button>
      </form>
      <p>
        Нет учебной записи? <Link href="/auth/register">Регистрация</Link>
      </p>
    </main>
  );
}
```

Файл app/auth/register/page.tsx

```
// Подключает форму регистрации к POST /api/auth/register.

"use client";

import Link from "next/link";
import { useRouter } from "next/navigation";
import { useState, type FormEvent } from "react";
import { useAuth } from "@/app/providers/AuthProvider";

export default function RegisterPage() {
  const router = useRouter();
  const { register } = useAuth();
  const [error, setError] = useState("");
  const [pending, setPending] = useState(false);

  async function handleSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault();
    setError("");
    setPending(true);

    const form = new FormData(event.currentTarget);

    try {
      // Формируем контракт регистрации из полей доступной HTML-формы.
      await register({
        name: String(form.get("name") ?? ""),
        email: String(form.get("email") ?? ""),
        password: String(form.get("password") ?? ""),
        confirmPassword: String(form.get("confirmPassword") ?? ""),
```

```
      });
      router.push("/gallery");
    } catch (value) {
      setError(value instanceof Error ? value.message : "Не удалось
зарегистрироваться");
    } finally {
      setPending(false);
    }
  }

  return (
    <main className="form-page">
      <h1>Регистрация</h1>
      <form className="auth-form" onSubmit={handleSubmit}>
        <label htmlFor="name">Имя пользователя</label>
        <input id="name" name="name" autoComplete="name" required />
        <label htmlFor="email">Email</label>
        <input id="email" name="email" type="email" autoComplete="email"
required />
        <label htmlFor="password">Пароль</label>
        <input
          id="password"
          name="password"
          type="password"
          autoComplete="new-password"
          minLength={8}
          required
        />
        <label htmlFor="confirmPassword">Повторите пароль</label>
        <input
          id="confirmPassword"
          name="confirmPassword"
          type="password"
          autoComplete="new-password"
          minLength={8}
          required
        />

        {error ? <p role="alert" className="form-error">{error}</p> : null}
        <button type="submit" disabled={pending}>
          {pending ? "Регистрация…" : "Зарегистрироваться"}
        </button>
      </form>
      <p>
        Уже есть учебная запись? <Link href="/auth/login">Войти</Link>
      </p>
    </main>
  );
}
```

Файл app/gallery/_components/SignalsGallery.tsx

```
// Загружает реальные сигналы и сохраняет отслеживание через серверный API.

"use client";

import Image from "next/image";
import Link from "next/link";
import { useCallback, useEffect, useState } from "react";
import PhotoMap from "./PhotoMap";
import PhotoModal from "./PhotoModal";
import { useAuth } from "@/app/providers/AuthProvider";
import styles from "./SignalsGallery.module.css";
```

```
export type NeighborSignalDTO = {
  id: string;
  slug: string;
  title: string;
  summary: string;
  category: string;
  location: string;
  image: string;
  imageAlt: string;
  latitude: number;
  longitude: number;
  isTracked: boolean;
};

export default function SignalsGallery() {
  const { status, refresh } = useAuth();
  const [signals, setSignals] = useState<NeighborSignalDTO[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");
  const [pendingId, setPendingId] = useState<string | null>(null);
  const [reloadKey, setReloadKey] = useState(0);
  const [selected, setSelected] = useState<NeighborSignalDTO | null>(null);
  const [returnFocusTo, setReturnFocusTo] = useState<HTMLButtonElement |
null>(null);

  function openFromCard(signal: NeighborSignalDTO, button: HTMLButtonElement)
{
    setReturnFocusTo(button);
    setSelected(signal);
  }

  const openFromMap = useCallback((signal: NeighborSignalDTO) => {
    setReturnFocusTo(null);
    setSelected(signal);
  }, []);

  function closeModal() {
    setSelected(null);
  }

  // Повторная загрузка вызывается кнопкой после сетевой ошибки.
  const retry = useCallback(() => setReloadKey((value) => value + 1), []);

  useEffect(() => {
    const controller = new AbortController();

    async function loadSignals() {
      setLoading(true);
      setError("");

      try {
        const response = await fetch("/api/signals", {
          cache: "no-store",
          credentials: "same-origin",
          signal: controller.signal,
        });

        if (!response.ok) throw new Error("Не удалось загрузить сигналы");
```

```
        const payload = (await response.json()) as { signals:
NeighborSignalDTO[] };
        setSignals(payload.signals);
      } catch (value) {
        // Игнорируем ожидаемую отмену при размонтировании компонента.
        if (value instanceof DOMException && value.name === "AbortError")
return;
        setError(value instanceof Error ? value.message : "Ошибка загрузки");
      } finally {
        if (!controller.signal.aborted) setLoading(false);
      }
    }

    void loadSignals();

    // Отменяем запрос, если компонент больше не используется.
    return () => controller.abort();
  }, [reloadKey, status]);

  async function toggleTracked(signal: NeighborSignalDTO) {
    if (status !== "authenticated") return;

    setPendingId(signal.id);
    setError("");

    try {
      const response = await fetch(`/api/favorites/${signal.id}`, {
        method: signal.isTracked ? "DELETE" : "POST",
        credentials: "same-origin",
      });

      if (response.status === 401) {
        // Синхронизируем UI, если сервер больше не принимает сессию.
        await refresh();
        throw new Error("Сеанс завершён. Выполните вход снова.");
      }

      if (!response.ok) throw new Error("Не удалось изменить отслеживание");

      const nextTracked = !signal.isTracked;

      // Меняем только карточку, изменение которой подтвердил сервер.
      setSignals((current) =>
        current.map((item) =>
          item.id === signal.id
            ? { ...item, isTracked: nextTracked }
            : item,
        ),
      );
      setSelected((current) =>
        current && current.id === signal.id
          ? { ...current, isTracked: nextTracked }
          : current,
      );
    } catch (value) {
      setError(value instanceof Error ? value.message : "Ошибка
отслеживания");
    } finally {
      setPendingId(null);
    }
  }
```

```
  if (loading) return <p role="status">Загрузка сигналов…</p>;

  if (error && signals.length === 0) {
    return (
      <div role="alert">
        <p>{error}</p>
        <button type="button" onClick={retry}>Повторить</button>
      </div>
    );
  }

  if (signals.length === 0) return <p>Сигналы пока не добавлены.</p>;

  return (
    <>
      {error ? <p role="alert">{error}</p> : null}

      <div className={styles.grid}>
        {signals.map((signal, index) => (
          <article className={styles.card} key={signal.id}>
            <button
              className={styles.cardOpen}
              type="button"
              onClick={(event) => openFromCard(signal, event.currentTarget)}
            >
              <span className={styles.imageWrap}>
                <Image
                  src={signal.image}
                  alt={signal.imageAlt}
                  fill
                  sizes="(max-width: 47.99rem) calc(100vw - 2rem), (max-
width: 74.99rem) 45vw, 22rem"
                  className={styles.image}
                  // Первая карточка является LCP-изображением первого
экрана.
                  loading={index === 0 ? "eager" : "lazy"}
                  fetchPriority={index === 0 ? "high" : "auto"}
                />
              </span>
              <span className={styles.cardBody}>
                <span className={styles.title}>{signal.title}</span>
                <span className={styles.summary}>{signal.summary}</span>
              </span>
            </button>

            {status === "authenticated" ? (
              <button
                type="button"
                className={styles.trackButton}
                disabled={pendingId === signal.id}
                aria-pressed={signal.isTracked}
                aria-label={`${
                  signal.isTracked
                    ? "Удалить из отслеживания"
                    : "Добавить в отслеживание"
                }: ${signal.title}`}
                onClick={() => void toggleTracked(signal)}
              >
                {signal.isTracked ? "В отслеживании" : "Отслеживать"}
              </button>
            ) : (
```

```
              <Link className={styles.trackLink} href="/auth/login">
                Войдите, чтобы отслеживать сигнал
              </Link>
            )}
          </article>
        ))}
      </div>

      <PhotoMap signals={signals} onSelect={openFromMap} />

      {selected ? (
        <PhotoModal
          signal={selected}
          pending={pendingId === selected.id}
          onToggleTracked={() => void toggleTracked(selected)}
          onClose={closeModal}
          returnFocusTo={returnFocusTo}
        />
      ) : null}
    </>
  );
}
```

Файл app/layout.tsx

```
// Подключает реальный AuthProvider вместо учебного DemoAuthProvider.

import type { Metadata, Viewport } from "next";
import type { ReactNode } from "react";
import AuthProvider from "./providers/AuthProvider";
import BurgerMenu from "./ui/BurgerMenu/BurgerMenu";
import "./globals.css";

export const metadata: Metadata = {
  title: "SignalBoard",
  description: "Доска локальных сигналов соседям",
};

export const viewport: Viewport = {
  themeColor: "#eef3f8",
  colorScheme: "light",
};

export default function RootLayout({ children }: { children: ReactNode }) {
  // Провайдер обслуживает только клиентские потребители auth-состояния.
  return (
    <html lang="ru">
      <body>
        <AuthProvider>
          <BurgerMenu />
          {children}
        </AuthProvider>
      </body>
    </html>
  );
}
```

Файл vitest.config.mts

```
// Настраивает быстрые unit tests чистой серверной логики.

import { defineConfig } from "vitest/config";
import tsconfigPaths from "vite-tsconfig-paths";
```

```
export default defineConfig({
  // Поддерживаем алиасы путей из tsconfig.json.
  plugins: [tsconfigPaths()],
  test: {
    // Чистая валидация не требует DOM или браузера.
    environment: "node",
    include: ["tests/unit/**/*.test.ts"],
  },
});
```

Файл tests/unit/validation.test.ts

```
// Проверяет чистую нормализацию и server-side validation без базы.

import { describe, expect, test } from "vitest";
import {
  normalizeEmail,
  validateRegistration,
} from "@/app/lib/validation";

describe("normalizeEmail", () => {
  test("удаляет пробелы и приводит email к нижнему регистру", () => {
    expect(normalizeEmail("  Neighbor@Example.TEST ")).toBe(
      "neighbor@example.test",
    );
  });
});

describe("validateRegistration", () => {
  test("принимает корректный набор полей", () => {
    const result = validateRegistration({
      name: "Сосед",
      email: "neighbor@example.test",
      password: "SignalPass_2026!",
      confirmPassword: "SignalPass_2026!",
    });

    expect(result.errors).toEqual([]);
    expect(result.data?.email).toBe("neighbor@example.test");
  });

  test("отклоняет короткий пароль", () => {
    const result = validateRegistration({
      name: "Сосед",
      email: "neighbor@example.test",
      password: "short",
      confirmPassword: "short",
    });

    expect(result.data).toBeNull();
    expect(result.errors).toContain(
      "Пароль должен содержать не менее 8 символов",
    );
  });

  test("отклоняет несовпадающее подтверждение", () => {
    const result = validateRegistration({
      name: "Сосед",
      email: "neighbor@example.test",
      password: "SignalPass_2026!",
      confirmPassword: "Different_2026!",
```

```
    });

    expect(result.errors).toContain("Пароли не совпадают");
  });
});
```

Файл playwright.config.ts

```
// Запускает E2E против production-сборки SignalBoard.

import "dotenv/config";
import { defineConfig, devices } from "@playwright/test";

export default defineConfig({
  testDir: "./tests/e2e",
  // Общая локальная база проверяется последовательно.
  workers: 1,
  use: {
    baseURL: "http://127.0.0.1:3000",
    trace: "on-first-retry",
    ...devices["Desktop Chrome"],
  },
  webServer: {
    // Перед test:e2e уже должна существовать npm run build.
    command: "npm run start",
    url: "http://127.0.0.1:3000",
    reuseExistingServer: !process.env.CI,
    timeout: 120_000,
  },
});
```

Файл tests/e2e/auth-and-favorites.spec.ts

```
// Проверяет регистрацию, cookie-сессию и сохранение отслеживания после
reload.

import "dotenv/config";
import { PrismaClient } from "@prisma/client";
import { expect, test } from "@playwright/test";

const prisma = new PrismaClient();
let testEmail = "";

test.afterAll(async () => {
  // Удаляем только пользователя, созданного этим тестом.
  if (testEmail) {
    await prisma.user.deleteMany({ where: { email: testEmail } });
  }
  await prisma.$disconnect();
});

test("пользователь регистрируется и сохраняет сигнал в отслеживание", async
({
  page,
}) => {
  // Уникальный email исключает конфликт с предыдущим локальным запуском.
  testEmail = `e2e-${Date.now()}@example.test`;

  await page.goto("/auth/register");
  await page.getByLabel("Имя пользователя").fill("E2E сосед");
  await page.getByLabel("Email").fill(testEmail);
  await page.getByLabel("Пароль", { exact: true }).fill("SignalPass_2026!");
  await page.getByLabel("Повторите пароль").fill("SignalPass_2026!");
```

```
  await page.getByRole("button", { name: "Зарегистрироваться" }).click();

  // После успешного API-ответа форма переводит пользователя в галерею.
  await expect(page).toHaveURL(/\/gallery$/);

  const addTracked = page.getByRole("button", {
    name: "Добавить в отслеживание: Ищу ключи от подъезда",
  });
  await addTracked.click();

  // Сервер подтвердил изменение, поэтому доступно действие удаления.
  await expect(
    page.getByRole("button", {
      name: "Удалить из отслеживания: Ищу ключи от подъезда",
    }),
  ).toBeVisible();

  // Перезагрузка повторно читает cookie и TrackedSignal из PostgreSQL.
  await page.reload();
  await expect(
    page.getByRole("button", {
      name: "Удалить из отслеживания: Ищу ключи от подъезда",
    }),
  ).toBeVisible();

  // Завершаем сессию через тот же browser context.
  await page.request.post("/api/auth/logout");
  await page.reload();
  await expect(
    page.getByRole("link", { name: "Войдите, чтобы отслеживать сигнал"
}).first(),
  ).toBeVisible();
});
```